Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №3 по курсу

«Операционные системы»

Группа: М8О-212БВ-24

Студент: Колосов Е. Е.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 31.10.25

Москва, 2025

Постановка задачи

Вариант 5.

Составить и отладить программу на языке Си, осуществляющую работу с процессами и взаимодействие между ними в одной из двух операционных систем. В результате работы программа (основной процесс) должен создать для решения задачи один или несколько дочерних процессов. Взаимодействие между процессами осуществляется через именованную разделяемую память (POSIX shared memory) и семафоры для синхронизации. Необходимо обрабатывать системные ошибки, которые могут возникнуть в результате работы.

Общий метод и алгоритм решения

Использованные системные вызовы:

1. shm_open(const char *name, int oflag, mode_t mode)
    - Назначение: создаёт или открывает именованный объект разделяемой памяти (shared memory object), находящийся в оперативной памяти, а не на диске.
2. shm_unlink(const char *name)
    - Назначение: удаляет именованный объект разделяемой памяти из системы.
3. ftruncate(int fd, off_t length)
    - Назначение: устанавливает размер объекта разделяемой памяти (аналогично файлу).
4. mmap(void *addr, size_t length, int prot, int flags, int fd, off_t offset)
    - Назначение: отображает разделяемую память в адресное пространство процесса, позволяя работать с ней как с обычной переменной.
5. munmap(void *addr, size_t length)
    - Назначение: отменяет отображение памяти, созданное через mmap.
6. sem_open(const char *name, int oflag, mode_t mode, unsigned int value)
    - Назначение: создаёт или открывает именованный семафор POSIX для синхронизации доступа к разделяемой памяти.
7. sem_wait(sem_t *sem)
    - Назначение: ожидает, пока семафор не станет положительным, затем уменьшает его значение (блокирующий вызов).
8. sem_post(sem_t *sem)
    - Назначение: увеличивает значение семафора, сигнализируя другому процессу, что можно продолжить работу.
9. sem_close(sem_t *sem)
    - Назначение: закрывает дескриптор семафора.
10. sem_unlink(const char *name)
    - Назначение: удаляет именованный семафор из системы.

11. fork(void)
    - Назначение: создаёт дочерний процесс, который является копией родительского.
12. execl(const char *path, const char *arg0, ..., (char *)NULL)
    - Назначение: заменяет образ текущего процесса на новый исполняемый файл.
13. read(int fd, void *buf, size_t count)
    - Назначение: читает данные из файлового дескриптора (ввод с клавиатуры в родительском процессе).
14. write(int fd, const void *buf, size_t count)
    - Назначение: записывает данные в файловый дескриптор (например, вывод ошибок или в файл).
15. open(const char *pathname, int flags, mode_t mode)
    - Назначение: открывает файл (или создаёт, если его нет), в котором дочерний процесс записывает числа.
16. close(int fd)
    - Назначение: закрывает файловый дескриптор.
17. wait(pid_t pid)
    - Назначение: ожидает завершения дочернего процесса.
18. exit(int status)
    - Назначение: завершает процесс с указанным статусом.

Описание реализации:

В рамках лабораторной работы была реализована программа, состоящая из двух процессов — родительского и дочернего, взаимодействующих через именованную разделяемую память (POSIX shared memory) и семафоры для синхронизации. Программа написана на языке C с использованием только системных вызовов POSIX, без библиотек stdio.h, threads.h и stdatomic.h.

Родительский процесс считывает числа из stdin и записывает их в разделяемую память, после чего сигнализирует дочернему процессу с помощью семафора. Дочерний процесс ожидает сигнала, читает число из разделяемой памяти, проверяет его на простоту: составные числа записываются в файл, а при получении простого или отрицательного числа оба процесса завершаются. Взаимодействие между процессами обеспечивается с помощью системных вызовов shm_open, ftruncate, mmap, munmap, sem_open, sem_wait, sem_post, sem_close, sem_unlink, fork, execl, read, write, open, close, wait и exit. Программа корректно обрабатывает числа и завершает работу в соответствии с условиями задания.

Код программы

Parent.c

```
#include <stdlib.h>

#include <unistd.h>

#include <sys/wait.h>

#include <sys/mman.h>

#include <fcntl.h>

#include <semaphore.h>

#include <string.h>


int IsPrime(int number) {

    if (number <= 1) return 0;

    if (number == 2) return 1;

    if (number % 2 == 0) return 0;

    for (int i = 3; i * i <= number; i += 2) {

        if (number % i == 0) return 0;

    }

    return 1;

}


void int_to_string(int n, char* buffer) {

    if (n == 0) {
```

```
        buffer[0] = '0';
        buffer[1] = '\0';
        return;
    }

    int temp = n;
    int len = 0;
    if (temp < 0) {
        buffer[0] = '-';
        temp = -temp;
        len = 1;
    }

    int start = len;
    while (temp > 0) {
        buffer[len++] = (temp % 10) + '0';
        temp /= 10;
    }
    buffer[len] = '\0';

    int end = len - 1;
    while (start < end) {
        char t = buffer[start];
        buffer[start] = buffer[end];
        buffer[end] = t;
        start++;
        end--;
    }
}
```

```
int main() {
    char pid_str[16];
    int_to_string(getpid(), pid_str);

    char shm_name[64];
    strcpy(shm_name, "/shm_");
    strcat(shm_name, pid_str);

    char sem_name[64];
    strcpy(sem_name, "/sem_");
    strcat(sem_name, pid_str);

    int shm_fd = shm_open(shm_name, O_CREAT | O_RDWR, 0666);
    if (shm_fd == -1) {
        const char error_message[] = "shm_open failed\n";
        write(STDERR_FILENO, error_message, sizeof(error_message) - 1);
        exit(1);
    }

    if (ftruncate(shm_fd, sizeof(int)) == -1) {
        const char error_message[] = "ftruncate failed\n";
        write(STDERR_FILENO, error_message, sizeof(error_message) - 1);
        exit(1);
    }

    int* shared_num = (int*)mmap(NULL, sizeof(int), PROT_READ | PROT_WRITE, MAP_SHARED,
shm_fd, 0);
    if (shared_num == MAP_FAILED) {
        const char error_message[] = "mmap failed\n";
        write(STDERR_FILENO, error_message, sizeof(error_message) - 1);
```

```
    exit(1);
}

sem_t* sem = sem_open(sem_name, O_CREAT, 0666, 0);
if (sem == SEM_FAILED) {
    const char error_message[] = "sem_open failed\n";
    write(STDERR_FILENO, error_message, sizeof(error_message) - 1);
    exit(1);
}

char fileName[256];
const char prompt[] = "Enter filename: ";
write(STDOUT_FILENO, prompt, sizeof(prompt) - 1);

int index = 0;
char character;
while (read(STDIN_FILENO, &character, 1) == 1 && character != '\n') {
    fileName[index++] = character;
}
fileName[index] = '\0';

pid_t processId = fork();
if (processId == 0) {
    execl("./child", "child", fileName, shm_name, sem_name, NULL);
    const char exec_error[] = "exec failed\n";
    write(STDERR_FILENO, exec_error, sizeof(exec_error) - 1);
    exit(1);
} else if (processId > 0) {
    int number = 0;
    int sign = 1;
```

```
    char inputChar;

    while (read(STDIN_FILENO, &inputChar, 1) == 1) {
        if (inputChar == '-') {
            sign = -1;
        } else if (inputChar >= '0' && inputChar <= '9') {
            number = number * 10 + (inputChar - '0');
        } else if (inputChar == '\n') {
            number *= sign;

            *shared_num = number;
            sem_post(sem);

            if (number < 0 || IsPrime(number)) {
                break;
            }
            number = 0;
            sign = 1;
        }
    }

    wait(NULL);

    munmap(shared_num, sizeof(int));
    sem_close(sem);
    shm_unlink(shm_name);
    sem_unlink(sem_name);
} else {
    const char fork_error[] = "fork failed\n";
    write(STDERR_FILENO, fork_error, sizeof(fork_error) - 1);
```

```
        exit(1);
    }

    return 0;
}
```

Child.c

```
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <semaphore.h>
#include <string.h>

void int_to_string(int n, char* buffer) {
    if (n == 0) {
        buffer[0] = '0';
        buffer[1] = '\0';
        return;
    }

    int temp = n;
    int len = 0;
    if (temp < 0) {
        buffer[0] = '-';
        temp = -temp;
        len = 1;
    }

    int start = len;
```

```
    while (temp > 0) {
        buffer[len++] = (temp % 10) + '0';
        temp /= 10;
    }
    buffer[len] = '\0';

    int end = len - 1;
    while (start < end) {
        char t = buffer[start];
        buffer[start] = buffer[end];
        buffer[end] = t;
        start++;
        end--;
    }
}

int IsPrime(int number) {
    if (number <= 1) return 0;
    if (number == 2) return 1;
    if (number % 2 == 0) return 0;
    for (int i = 3; i * i <= number; i += 2) {
        if (number % i == 0) return 0;
    }
    return 1;
}

int main(int argc, char* argv[]) {
    if (argc != 4) {
        const char usage_message[] = "Usage: child <output_file> <shm_name> <sem_name>\n";
        write(STDERR_FILENO, usage_message, sizeof(usage_message) - 1);
```

```
        exit(1);
    }

    const char* output_file = argv[1];
    const char* shm_name = argv[2];
    const char* sem_name = argv[3];

    int shm_fd = shm_open(shm_name, O_RDWR, 0666);
    if (shm_fd == -1) {
        const char open_error[] = "shm_open failed\n";
        write(STDERR_FILENO, open_error, sizeof(open_error) - 1);
        exit(1);
    }

    int* shared_num = (int*)mmap(NULL, sizeof(int), PROT_READ | PROT_WRITE, MAP_SHARED,
shm_fd, 0);
    if (shared_num == MAP_FAILED) {
        const char mmap_error[] = "mmap failed\n";
        write(STDERR_FILENO, mmap_error, sizeof(mmap_error) - 1);
        exit(1);
    }

    sem_t* sem = sem_open(sem_name, 0);
    if (sem == SEM_FAILED) {
        const char sem_error[] = "sem_open failed\n";
        write(STDERR_FILENO, sem_error, sizeof(sem_error) - 1);
        exit(1);
    }

    int fileDescriptor = open(output_file, O_WRONLY | O_CREAT | O_TRUNC, 0644);
```

```
if (fileDescriptor == -1) {
    const char open_error[] = "open failed\n";
    write(STDERR_FILENO, open_error, sizeof(open_error) - 1);
    exit(1);
}

while (1) {
    sem_wait(sem);

    int number = *shared_num;

    // Проверяем, нужно ли завершить, **до** записи
    if (number < 0 || IsPrime(number)) {
        break;
    }

    // Преобразуем число в строку
    char buffer[32];
    int temp = number;
    int index = 0;

    if (temp == 0) {
        buffer[index++] = '0';
    } else {
        if (temp < 0) {
            buffer[index++] = '-';
            temp = -temp;
        }
        int start = index;
        while (temp > 0) {
```

```
            buffer[index++] = (temp % 10) + '0';
            temp /= 10;
        }
        int end = index - 1;
        while (start < end) {
            char t = buffer[start];
            buffer[start] = buffer[end];
            buffer[end] = t;
            start++;
            end--;
        }
    }

    buffer[index++] = '\n';
    write(fileDescriptor, buffer, index);
  }

  close(fileDescriptor);
  munmap(shared_num, sizeof(int));
  sem_close(sem);
  return 0;
}
```

Протокол работы программы

```
./parent
Enter filename: output.txt
10
15
-5
```

```
cat output.txt

10

15
```

Вывод

В ходе лабораторной работы была реализована программа, обеспечивающая взаимодействие между родительским и дочерним процессами через именованную разделяемую память (POSIX shared memory) и семафоры для синхронизации. Программа корректно обрабатывает числа, записывает составные в файл и завершает работу при получении простого или отрицательного числа. Все требования задания выполнены, использованы только системные вызовы POSIX. Работа показала, как можно организовать эффективное межпроцессное взаимодействие в операционных системах на базе Unix без использования каналов (pipe).